

Types for Programs and Proofs

The identity type, and indexed inductive types

Thierry Coquand

Plan

First hour, `Id5.agda`: the *identity type*, and *elimination*

- the *motive* of an elimination rule: `BoolRec`, `NatRec`, `listRec`
- the same rule for `Id`, which is *J*; and extensionality

Second hour, `deduction.agda`: *indexed inductive types*

- parameters and indices; `Vec`, `Fin`, and their eliminators
- two derivations of the transitivity of $\leq$; the deduction theorem

Where we are

Andreas presented type theory as *judgment forms* and *inference rules*, with five principles per type former: formation, introduction, elimination, computation, extensionality

For the identity type (Martin-Löf, 1973) he gave formation, introduction, and *one* elimination rule

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash \text{Id } A \ a \ b \text{ type}} \qquad \frac{\Gamma \vdash a : A}{\Gamma \vdash \text{refl } a : \text{Id } A \ a \ a}$$

Today: the *general* elimination rule, and the question of whether there is anything more to say about a proof of an equality

Elimination rules and their motive

Every elimination rule has the same shape

- a *motive* C : what we want to prove, which may depend on the element being eliminated
- one premise per *constructor*, at the corresponding instance of C
- the conclusion: the statement $(x:A) \rightarrow C\ x$ itself

Constant motive: a definition by recursion. Motive depending on the element: a proof by *induction*. Same rule, same computation rule — a program and a proof by induction are the same object

BoolRec

```
BoolRec : (C : Bool -> Set) -> C true -> C false -> (b : Bool) -> C b
BoolRec C x y true  = x
BoolRec C x y false = y
```

With C constant this is `if_then_else_`. With C depending on b we can prove things:

```
notnot : (b : Bool) -> Id Bool (not (not b)) b
notnot = BoolRec (\ b -> Id Bool (not (not b)) b) (refl true) (refl false)
```

The two premises are `Id Bool (not (not true)) true` and the same at false; both *compute* to `Id Bool true true`, proved by `refl true`

NatRec

```
NatRec : (C : Nat -> Set) ->
  C zero ->
  ((n : Nat) -> C n -> C (suc n)) ->
  (n : Nat) -> C n
```

```
addZero : (n : Nat) -> Id Nat (n + zero) n
addZero = NatRec (\ n -> Id Nat (n + zero) n) (refl zero) (\ n h -> cong suc h)
```

Constant motive: primitive recursion. Motive depending on n : mathematical induction — base case, induction step, and the recursive call *is* the induction hypothesis

Recall $n + \text{zero}$ does not compute; this is the proof by induction that it equals n

listRec

```
listRec : {A : Set} (C : list A -> Set) ->
  C nil ->
  ((a : A) (as : list A) -> C as -> C (cons a as)) ->
  (as : list A) -> C as
listRec C base step nil          = base
listRec C base step (cons a as) = step a as (listRec C base step as)
```

The same shape once more: one premise per constructor, each at the corresponding instance of the motive

We shall write exactly the same rule for `Vec`, for $\leq$, and for the derivations $\Gamma \vdash X$ of the second hour

Binary trees

```
data Tree (A : Set) : Set where
  leaf    : A -> Tree A
  branch  : Tree A -> Tree A -> Tree A

TreeRec : {A : Set} (C : Tree A -> Set) ->
  ((a : A) -> C (leaf a)) ->
  ((t1 t2 : Tree A) -> C t1 -> C t2 -> C (branch t1 t2)) ->
  (t : Tree A) -> C t
TreeRec C base step (leaf a)          = base a
TreeRec C base step (branch t1 t2) =
  step t1 t2 (TreeRec C base step t1) (TreeRec C base step t2)
```

branch has *two* recursive arguments, so the step case has two induction hypotheses — and the definition makes two recursive calls

The elimination rule of `Id`

To prove $(a\ b:A) \rightarrow (p:\text{Id } A\ a\ b) \rightarrow C\ a\ b\ p$ it is enough to treat the case of the one constructor, `refl` x

$$\frac{\Gamma \vdash C : (x\ y:A) \rightarrow \text{Id } A\ x\ y \rightarrow \text{Set} \quad \Gamma \vdash d : (x:A) \rightarrow C\ x\ x\ (\text{refl } x)}{\Gamma \vdash J\ C\ d : (a\ b:A) \rightarrow (p:\text{Id } A\ a\ b) \rightarrow C\ a\ b\ p}$$

The motive mentions the two endpoints *and* the proof; the premise is a *function*, the case of `refl` for an arbitrary x

Exactly the shape of the rules of the last four slides, with the motive of `Id`

The identity type in Agda

In Agda the type is declared as an *inductive family*

```
data Id (A : Set) : A -> A -> Set where
  refl : (a : A) -> Id A a a
```

- *A*, left of the colon, is a *parameter*: the same in every conclusion
- the two endpoints, right of it, are *indices*: *refl a* chooses both of them to be *a*

These are exactly the rules above — and the identity type is also our first example of the subject of the second hour

Another version of the identity type

One may instead take the first endpoint as a *parameter*: it is then fixed, and only the second one is an index

```
data Id' (A : Set) (a : A) : A -> Set where  
  refl : Id' A a a
```

This is the version of the last lecture. Its elimination rule has a motive over $y:A$ and $q : \text{Id}' A a y$, with a *fixed*

The two versions are *equivalent*. From the rule of Id' the general rule follows at once, by keeping a fixed in the motive

It is the *converse* — deriving the rule of Id' from the general rule — which is surprisingly subtle, and we do not do it in this lecture

The eliminator of Id

```
IdRec : {A : Set} (C : (x y : A) -> Id A x y -> Set) ->
        ((x : A) -> C x x (refl x)) ->
        (a b : A) (p : Id A a b) -> C a b p
IdRec C d a a (refl a) = d a
```

A motive, one premise for the one constructor `refl`, then the statement to be proved: the shape of `BoolRec`, `NatRec`, `listRec`

Its traditional name is *J*. Computation rule, the equation of the definition:
 $\text{IdRec } C \, d \, a \, a \, (\text{refl } a) = d \, a$

And **subst** is the instance whose motive ignores the proof:

```
subst C = J (\ x y _ -> C x -> C y) (\ x w -> w)
```

How to read J

In words: *to prove a statement about an arbitrary proof of an equality, it is enough to prove it when the proof is `refl x`*

Note what J does *not* say

- it does *not* say that p is `refl`
- it does *not* say that a and b are the same variable
- the premise is only that C holds at `refl x`, for an arbitrary x

Same discipline as for the other eliminators: one case per constructor — and `Id` has one constructor

subst, the restricted form

Take the motive to *ignore the proof*, and to depend on *one* endpoint only: we obtain the rule we already know

$$\frac{\Gamma \vdash C : A \rightarrow \text{Set}}{\Gamma \vdash \text{subst } C : (a\ b:A) \rightarrow \text{Id } A\ a\ b \rightarrow C\ a \rightarrow C\ b}$$

```
subst : {A : Set} (C : A -> Set) (a b : A) -> Id A a b -> C a -> C b
subst C = J (\ x y _ -> C x -> C y) (\ x w -> w)
```

```
sym {A} a b p = subst (\ y -> Id A y a) a b p (refl a)
trans {A} a b c p q = subst (\ y -> Id A a y) b c q p
```

Leibniz' principle: equals may be substituted for equals

Two equalities

 $\Gamma \vdash t = u : A$ a *judgment*

decided by the machine

used by `conv`, automatically

cannot be assumed or negated

sees only computation

 $\Gamma \vdash p : \text{Id } A \ t \ u$ a *type*

inhabited by a proof the user writes

used by `subst`, explicitly

can be assumed, negated, quantified

sees any proof, e.g. by induction

Recall the example: *zero + n computes* to *n*, but *n + zero* does not compute
— *n* is a variable

n + zero = n is not a judgmental equality; it is a *theorem*, proved by induction

Zero is not one

Nothing so far distinguishes two elements of `Nat`: `subst` and *`J`* only *replace* equals by equals

So define equality of numbers a second time, *by recursion*, so that it computes

```
eqNat : Nat -> Nat -> Set
eqNat  zero      zero    =  $\top$ 
eqNat  zero      (suc y) =  $\perp$ 
eqNat  (suc x)    zero    =  $\perp$ 
eqNat  (suc x)    (suc y) = eqNat x y
```

```
eqNatRefl : (x : Nat) -> eqNat x x
eqNatRefl  zero    = tt
eqNatRefl  (suc x) = eqNatRefl x
```

Zero is not one, and why one wants pattern matching

```
toEq    : (x y : Nat) -> Id Nat x y -> eqNat x y      -- subst, with
toEq x y p = subst (\ z -> eqNat x z) p (eqNatRefl x) -- motive eqNat x

fromEq : (x y : Nat) -> eqNat x y -> Id Nat x y      -- the converse

zeroNotOne : Id Nat zero (suc zero) -> ⊥
zeroNotOne p = toEq zero (suc zero) p
```

`Id Nat x y` and `eqNat x y` are *equivalent*, and `eqNat zero (suc zero)` computes to `⊥`: there is nothing more to do

With *pattern matching* the same proof is one line, `zeroNotOne ()`: no constructor can produce `Id Nat zero (suc zero)`, so Agda asks for no clause at all

Pattern matching, and eliminators

In Agda one writes definitions by *pattern matching*, not with eliminators — that is how BoolRec, NatRec, listRec, IdRec were defined

- each eliminator is defined by pattern matching, one line per constructor
- conversely a definition by pattern matching can be *translated* into eliminators, and Agda does this internally

That translation, and the conditions under which it is correct, is the work of *Jesper Cockx*

Pattern matching is what one writes; the eliminators say what the theory is

cong: a function respects equality

If a and b are equal then $f\ a$ and $f\ b$ are equal, for any function f

$$\frac{\Gamma \vdash f : A \rightarrow B}{\Gamma \vdash \text{cong } f : (a\ b:A) \rightarrow \text{Id } A\ a\ b \rightarrow \text{Id } B\ (f\ a)\ (f\ b)}$$

```
cong : {A B : Set} (f : A -> B) {a b : A} -> Id A a b -> Id B (f a) (f b)
cong f (refl a) = refl (f a)                                -- by pattern matching
```

```
cong {A} {B} f {a} {b} p =                                -- or as an instance of subst
  subst (\ y -> Id B (f a) (f y)) a b p (refl (f a))
```

The motive is $\lambda y. \text{Id } B\ (f\ a)\ (f\ y)$, proved at $y = a$ by $\text{refl } (f\ a)$

Equal functions are pointwise equal

```
happly : {A B : Set} (f g : A -> B) ->  
         Id (A -> B) f g -> (x : A) -> Id B (f x) (g x)  
happly f g p x = cong (\ h -> h x) p
```

Equal functions may be applied to the same argument: this is `cong`, that is Leibniz, at the function $h \mapsto h\ x$

So one direction is *provable*, and immediately so

It is the *converse* which is the question

Id is intensional

Two functions which agree on every argument, are they equal?

```
funExt : Set1
funExt = {A B : Set} (f g : A -> B) ->
  ((x : A) -> Id B (f x) (g x)) -> Id (A -> B) f g
```

The statement can be written down; it is *not* provable

Likewise *propositional extensionality*: two propositions which imply each other are equal

One may *add* them as axioms — and then these proofs no longer compute; or one may make equality *extensional*, a proof of *Id A a b* giving *a = b : A* — and then type checking is no longer decidable

Propositions as types?

A proposition is a *type* and a proof is an *element* of it

For equality this is a real decision: $\text{Id } A \ a \ b$ is a type with elements, not a truth value

- we can *state* properties of proofs, which the judgment $a = b : A$ cannot
- extensionality becomes a statement, which one may add or not
- whether two proofs of one equality are equal is not settled by the rules

These choices are what distinguishes the type theories in use today

Second hour: indexed inductive types

So far an inductive definition introduced *one* type

```
data list (A : Set) : Set where
  nil  : list A
  cons : A -> list A -> list A
```

Now *one type for each natural number*, all introduced at the same time

```
data Vec (A : Set) : Nat -> Set where
  []      : Vec A zero
  _::__   : {n : Nat} -> A -> Vec A n -> Vec A (suc n)
```

The two constructors land in *different* members of the family: `[]` in `Vec A zero` and `_::__` in `Vec A (suc n)`

Parameters and indices

parameter

written left of the colon
fixed once and for all
the same in every conclusion
`list A, Vec A ...`

index

written right of the colon
may vary from constructor to constructor
each constructor chooses its own
the length, the context, the formula

These are the *inductive families*; in proof theory one would say *inductively defined relations*

Everything of this second hour is in `deduction.agda`

Which cases have to be written?

```
head : {A : Set} {n : Nat} -> Vec A (suc n) -> A
head (a :: as) = a
```

The argument is of type `Vec A (suc n)`. Which constructor can have produced it?

- `[]` is of type `Vec A zero`, and `zero` is not `suc n`: *impossible*, there is no clause to write
- `_ :: _` is of type `Vec A (suc m)`: *possible*, reading `m` as `n`

Comparing the two *lengths* is all that happens, and Agda does it for us

One clause, and the definition is *complete*: no exception, no *Maybe*

What a case tells us

```
vadd : {n : Nat} -> Vec Nat n -> Vec Nat n -> Vec Nat n
vadd [] [] = []
vadd (a :: as) (b :: bs) = (a + b) :: vadd as bs
```

Two vectors of the *same* length, and only two clauses

Once the first argument is `[]` we know that `n` is zero; the second argument is then of type `Vec Nat zero`, so it *is* `[]` as well. The two mixed cases cannot be written down

Same thing in the first hour: `refl a` is of type `Id A a a`, so after matching a proof of `Id A a b` with `refl a` we know that `b` is `a`

The eliminator of Vec

```
VecRec : {A : Set} (C : (n : Nat) -> Vec A n -> Set) ->
  C zero [] ->
  ((n : Nat) (a : A) (as : Vec A n) -> C n as -> C (suc n) (a :: as)) ->
  (n : Nat) (as : Vec A n) -> C n as
VecRec C base step zero [] = base
VecRec C base step (suc n) (a :: as) = step n a as (VecRec C base step n as)
```

Exactly the shape of `listRec`. The only difference is the *motive*, which depends on the length as well as on the vector

An indexed family needs no new kind of rule — only a motive with more arguments

Finite types

```
data Fin : Nat -> Set where
  zero : {n : Nat} ->          Fin (suc n)
  suc   : {n : Nat} -> Fin n -> Fin (suc n)

lookup : {A : Set} {n : Nat} -> Vec A n -> Fin n -> A
lookup (a :: as) zero      = a
lookup (a :: as) (suc i)   = lookup as i
```

`Fin n` has exactly *n* elements; `Fin zero` is empty, since no constructor produces it

`lookup` is *total*: an index out of bounds cannot be written down. Two clauses, and again the impossible cases need not be mentioned

Relations defined inductively

```
data _<=_ : Nat -> Nat -> Set where
  leZero : {n : Nat} ->          zero <= n
  leSuc   : {m n : Nat} -> m <= n -> suc m <= suc n

leTrans : {l m n : Nat} -> l <= m -> m <= n -> l <= n
leTrans leZero      q      = leZero
leTrans (leSuc p)   (leSuc q) = leSuc (leTrans p q)
```

Here there is *no parameter*: both arguments are indices

A *first derivation* of transitivity, by induction on the derivation

Matching the second argument with `leSuc q` uses that a derivation of *$\text{suc } m \leq n$* can only be a `leSuc`: an *inversion*, done for us

The eliminator of $\leq$, and a motive

```
leRec : (R : Nat -> Nat -> Set) ->
        ((n : Nat) -> R zero n) ->
        ((x y : Nat) -> R x y -> R (suc x) (suc y)) ->
        (x y : Nat) -> x <= y -> R x y
```

To prove transitivity with `leRec` we must *choose the motive*. Take

```
R : Nat -> Nat -> Set
R x y = (z : Nat) -> y <= z -> x <= z

Rbase : (n : Nat) -> R zero n
Rbase n z q = leZero
```

The base case is immediate. In the step case we are given `suc y <= z` and must know that `z` is a *successor*

The same relation, by recursion

```
le : Nat -> Nat -> Set
le zero    y      =  $\top$ 
le (suc x) zero   =  $\perp$ 
le (suc x) (suc y) = le x y
```

```
toLe      : (x y : Nat) -> x <= y -> le x y
fromLe    : (x y : Nat) -> le x y -> x <= y
```

le *computes*: le (suc y) zero is $\perp$, and le (suc y) (suc z) is le y z

The two definitions agree, and what was missing above is now free: from a proof of $\text{suc } y \leq z$ we read off, by cases on the *number* z , that z is $\text{suc } z'$ and that $y \leq z'$

The second derivation

```
Rstep : (x y : Nat) -> R x y -> R (suc x) (suc y)
Rstep x y h zero q = efq (toLe (suc y) zero q)
Rstep x y h (suc z) q = leSuc (h z (fromLe y z (toLe (suc y) (suc z) q)))

leTrans2 : (x y z : Nat) -> x <= y -> y <= z -> x <= z
leTrans2 x y z p q = leRec R Rbase Rstep x y p z q
```

The case analysis is on the *number* z , not on a derivation

Two derivations of one theorem: the first is shorter, the second says exactly which induction is made, and on which motive

Two families at once

```
data Even : Nat -> Set
data Odd  : Nat -> Set

data Even where
  evenZero : Even zero
  evenSuc  : {n : Nat} -> Odd n -> Even (suc n)

data Odd where
  oddSuc : {n : Nat} -> Even n -> Odd (suc n)
```

Inductive definitions may be *mutual*; what matters is that the premises are *smaller*, so that induction is justified

Exercise: prove that **Even** *n* and **Odd** *n* cannot both hold

Well-typed syntax

```
data Typ : Set where
  nat bool : Typ

El : Typ -> Set          -- what an object type means
El nat  = Nat
El bool = Bool

data Exp : Typ -> Set where
  lit      : Nat -> Exp nat
  blit     : Bool -> Exp bool
  plus     : Exp nat -> Exp nat -> Exp nat
  isZero   : Exp nat -> Exp bool
  ifte     : {a : Typ} -> Exp bool -> Exp a -> Exp a -> Exp a
```

Only *well-typed* expressions exist: `plus (blit true) (lit zero)` is rejected, and the two branches of `ifte` must have the *same* type *a*

An evaluator that cannot go wrong

```
eval : {a : Typ} -> Exp a -> El a
eval (lit n)      = n
eval (blit b)     = b
eval (plus e1 e2) = eval e1 + eval e2
eval (isZero e)   = isZeroNat (eval e)
eval (ifte b e1 e2) = If (eval b) (eval e1) (eval e2)
```

The *result type depends on the index*: matching `lit n` tells Agda that *a = nat*, so `El a` is `Nat` and the number `n` is accepted

No tags, no runtime type errors, and nothing to prove: the type of `eval` is the theorem “evaluation preserves types”

The deduction theorem

“as well known as integration by parts”

A. Turing, 1944

A *Hilbert system* for implication: two axiom schemes and one rule

$$K : X \rightarrow (Y \rightarrow X) \quad S : (X \rightarrow (Y \rightarrow Z)) \rightarrow ((X \rightarrow Y) \rightarrow (X \rightarrow Z))$$

$$\frac{\Gamma \vdash Y \rightarrow X \quad \Gamma \vdash Y}{\Gamma \vdash X} \text{ mp}$$

Theorem: if $\Gamma, X \vdash Y$ then $\Gamma \vdash X \rightarrow Y$

No rule *introduces* an implication; the theorem says we may argue as if one did

Formulae and contexts

```
data Form : Set where
  imply : Form -> Form -> Form
  atom  : Nat -> Form

context : Set          -- the terminology comes from AUTOMATH
context = list Form

X => Y = imply X Y      -- notation
G , X = cons X G

data isIn : context -> Form -> Set where
  zero : {X : Form} {G : context} -> isIn (G , X) X
  suc  : {X Y : Form} {G : context} -> isIn G X -> isIn (G , Y) X
```

`isIn G X` is `Fin` again, indexed by a context: an element is a *position* in Γ , which remembers *which* formula sits there

Derivations as an inductive family

```
data _ $\vdash$ _ : context -> Form -> Set where
  var : {X : Form} {G : context} -> isIn G X -> G  $\vdash$  X
  axK : {X Y : Form} {G : context} -> G  $\vdash$  X => (Y => X)
  axS : {X Y Z : Form} {G : context} ->
    G  $\vdash$  (X => (Y => Z)) => ((X => Y) => (X => Z))
  mp  : {Y X : Form} {G : context} ->
    G  $\vdash$  Y => X ->
    G  $\vdash$  Y ->
    -----
    G  $\vdash$  X
```

Indexed by *both* the context and the formula. An element is a *derivation tree*; note that the premise *Y* of mp does not occur in the conclusion

The proof, by induction on the derivation

$$\begin{array}{c} \text{dedthm} : \{G : \text{context}\} \{X Y : \text{Form}\} \rightarrow \\ G, X \vdash Y \rightarrow \\ \hline G \vdash X \Rightarrow Y \end{array}$$

```
dedthm (var x)      = dedlem3 x
dedthm axK          = dedlem2 axK
dedthm axS          = dedlem2 axS
dedthm (mp h h1)    = mp (mp axS (dedthm h)) (dedthm h1)
```

Four clauses, one per constructor: this *is* the induction principle of the family, used by pattern matching

Note the recursive calls in the last clause: the induction hypothesis is used on *both* premises, at a formula *C* that we never have to name

Where S and K come from

```
dedlem1 : {G : context} {X : Form} -> G ⊢ X => X
dedlem1 {G} {X} = mp (mp axS axK) (axK {Y = X => X})
```

```
dedlem2 : {G : context} {X Y : Form} -> G ⊢ Y -> G ⊢ X => Y
dedlem2 d = mp axK d
```

```
dedlem3 : {G : context} {X Y : Form} -> isIn (G , X) Y -> G ⊢ X => Y
dedlem3 zero      = dedlem1
dedlem3 (suc h)   = mp axK (var h)
```

The hypothesis itself needs $I = SKK$; a hypothesis we do not use needs K ; and the case of mp — from $\Gamma \vdash X \rightarrow (C \rightarrow Y)$ and $\Gamma \vdash X \rightarrow C$ get $\Gamma \vdash X \rightarrow Y$ — is *exactly* what S says

The two axioms are not arbitrary: they are what the induction needs

Curry–Howard, once more

Read a formula as a *type* and a derivation as a *term*

$\Gamma \vdash X$	a combinatory term of type X with free variables in Γ
var	a variable
axK, axS	the constants K and S
mp	application
dedthm	<i>bracket abstraction</i> $\lambda^*x.t$

dedthm is Schönfinkel's and Curry's algorithm for eliminating a variable from a combinatory term (1924, 1930), and the proof that it is correct is the deduction theorem

A proof-theoretic theorem and a compilation algorithm are the same object

Using it

```
dedcor2 : {G : context} {X Y Z : Form} ->
          G ⊢ (X => (Y => Z)) => (Y => (X => Z))
dedcor2 = dedthm (dedthm (dedthm
  (mp (mp (var (suc (suc zero))) (var zero)) (var zero))))
```

Three hypotheses are introduced, the proof is two applications of *modus ponens*, and the three `dedthm` discharge them

Compare with what one must write *without* the theorem, placing *S* and *K* by hand (`flipImp` in the file). Same statement; the second is a puzzle

But now a question: `dedcor2` is a *closed term* of the family $\vdash$ — what is its *normal form*?

The price of a convenient theorem

theorem	direct	via dedthm
flip $(X \rightarrow Y \rightarrow Z) \rightarrow Y \rightarrow X \rightarrow Z$	19	161
W $(X \rightarrow X \rightarrow Y) \rightarrow X \rightarrow Y$	11	59
composition $(A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow A \rightarrow C$	27	161
constant $A \rightarrow B \rightarrow C \rightarrow A$	7	29
apply $A \rightarrow (A \rightarrow B) \rightarrow B$	25	35

Number of `var/axK/axS/mp` nodes of the normal form, counted in Agda by a function `size (checksize.agda)`

Each `dedthm` replaces every `mp d1 d2` by `mp (mp axS d1') d2'` and recurses into both subtrees; nesting it three times multiplies

What to remember

- every elimination rule has a *motive*: constant motive, recursion; dependent motive, induction
- *J* is that rule for *Id*: to prove something about an arbitrary proof of an equality, prove it for *refl*
- *Id* is intensional: extensionality is a statement one may add, not a consequence
- an inductive family is given by parameters and *indices*; so one can write vectors of a given length, *Fin*, $\leq$, well-typed syntax, and *derivations*

Exercises

1. Prove $(b : \text{Bool}) \rightarrow \text{Id Bool } (\text{not } (\text{not } b)) \ b$ by pattern matching, and compare with `notnot`
2. Prove $(m \ n : \text{Nat}) \rightarrow \text{Id Nat } (m + \text{suc } n) \ (\text{suc } (m + n))$, once with `NatRec` and once by pattern matching
3. Prove $\text{Even } n \rightarrow \text{Odd } n \rightarrow \perp$
4. Add `pair : Exp a -> Exp b -> Exp (prod a b)` to the well-typed syntax, and extend `E1` and `eval`
5. Prove $\Gamma \vdash (X \rightarrow Y) \rightarrow (Y \rightarrow Z) \rightarrow (X \rightarrow Z)$ with `dedthm`, and by hand

References

- P. Martin-Löf. *An intuitionistic theory of types: predicative part*, 1975
- P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984
- J. Cockx. *Dependent Pattern Matching and Proof-Relevant Unification*. PhD thesis, KU Leuven, 2017
- A. Turing. *The Reform of Mathematical Notation and Phraseology*, 1944

Files: `Id5.agda`, `deduction.agda`, `checksize.agda`